

Building a Course Registration System

How I turned a stressful student experience into working software

Your Full Name • Student ID • English Writing & Presentation Skills

Lecturer's Name • October 7, 2026

1. Introduction

At the start of every semester, thousands of students at my university wait in front of their computers for the course registration page to open. When it finally opens, everyone clicks at the same time. After only a few minutes, the popular classes are full. Some students find out that two of their classes are at the same time, and many of them are not even sure if their registration has been saved. I have had this experience many times, and it was always stressful.

This problem gave me the idea for the project that I am going to describe in this report: a **Course Management and Registration System**. It is a website that I designed and built by myself between January and March 2026. Students can use it to register for classes and check their timetable and grades. At the same time, the university staff can use it to manage courses, semesters and classes.

In this report, I will first explain why I chose this project and what I wanted to achieve. Then, I will describe what the system does and how I built it. After that, I will talk about the two biggest problems I had and how I solved them. Finally, I will explain what I have learned from this project.

2. Why I Chose This Project

There are two main reasons why I chose this project. The first reason is personal. As a student, I know the problem very well, so I knew exactly what a good system should do. The second reason is related to my studies. I wanted a project that was big enough to practise everything I had learned in my IT courses, from designing a database to building a website.

Before I started writing any code, I set myself four goals:

1. to build a **complete** system with two separate parts, one for students and one for staff;
2. to make the system **fair**, so that a class never has more students than seats;
3. to make it **easy to use**, with clear messages when there is a problem;
4. to make it **easy to install**, so that anyone can start it with only one command.

3. What the System Does

The system has two “portals”, or areas: one for students and one for administrators (the university staff). Each user can only see the area which matches their role. Table 1 shows the main features of each area.

Student portal	Administrator portal
See personal information and study status	Add and update courses
Register for a class, or cancel it, with one click	Set the dates of each semester and the registration period
See a weekly timetable of all their classes	Open new classes and choose lecturers and rooms
Check their scores, grades and GPA	Check how many students have registered

Table 1: Main features of the two portals

For example, if a student wants to join a class, they just open the registration page, choose the class and click *Register*. Before the system accepts the request, it checks several rules. If a rule is broken, for example because the class is already full, the student sees a clear message that explains the problem. I also added a *dark mode*, because it is more comfortable for the eyes when students use the website at night.

4. How I Built It

4.1 The three main parts

Like most websites today, my system has three parts which work together, as you can see in Figure 1.

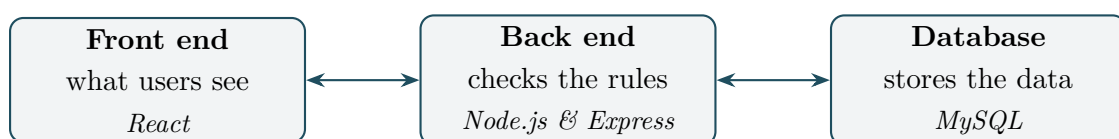


Figure 1: The three main parts of the system

The **front end** is the part that users see and click on in their browser. I built it with React, which is a popular tool for making websites. The **back end** runs on a server. It receives every request, checks who the user is, follows the registration rules and then sends back an answer. I wrote it in JavaScript with Node.js and Express. The last part is the **database**. It keeps all the information, such as students, courses, classes, timetables and grades, in eight tables which are connected to each other. For this part, I used MySQL.

To keep the system safe, every user has to log in, and passwords are never saved as normal text. In addition, I used a tool called *Docker*, so the whole system can be started with only

one command on any computer.

4.2 How I organised my work

I divided the work into five stages:

1. **Planning (January).** I looked at how registration works at my university, and I wrote down nine rules that the system must follow, for example “a student cannot register for a full class”.
2. **Database and back end (March).** I designed the database and built the back end, where all the rules are checked.
3. **Reorganising.** When the project became bigger, I changed the structure of the code, so the front end and the back end are now kept together in one tidy project.
4. **Front end.** I built the websites for students and staff, and then I improved the design.
5. **Getting ready to share.** I set up Docker and wrote guides that explain how to install and use the system.

During the whole project, I used Git, a tool which saves every version of the code. Thanks to Git, I could always go back if I made a mistake.

5. The Biggest Challenges

Most parts of the project were quite easy for me. However, two problems took much more time than I had expected.

5.1 Challenge 1: the last seat

Imagine a class with 40 seats, and 39 students have already registered. Now, two students click *Register* at exactly the same time. In a simple system, both requests ask the database “Is there a free seat?”. The database answers “Yes” to both of them, so both students are accepted. As a result, the class has 41 students, which is unfair and causes problems for the lecturer.

At first, I did not notice this problem, because it never happened when I was testing the system alone. However, after I read about how real registration systems work, I understood that it would certainly happen on a busy day. To solve it, I used a database feature called a **lock**. In simple words, when the first student starts to register for a class, the system “locks” that class for a very short moment. The second student has to wait until the first student has finished. When it is their turn, the system can see that the class is full, and it shows a polite message. I also added a rule inside the database itself. Because of this rule, the number of students can never be bigger than the number of seats, even if there is a mistake in my code.

5.2 Challenge 2: timetable clashes

The second problem was how to stop students from registering for two classes at the same time. It sounds simple, but real timetables are more complicated than they look. For example, some classes only meet in odd weeks, while others only meet in even weeks. In the end, I decided on a clear rule: two classes clash only if *all three* of these things are true:

- they are on the same day of the week;
- their lesson times overlap, even by one period;
- their weeks match (a class in odd weeks never clashes with a class in even weeks).

When the system finds a clash, it does not only say “Error”. Instead, it tells the student which of their classes is causing the problem, so they know what they need to change.

6. What I Have Learned

I have learned much more from this project than from any normal homework exercise. On the technical side, I now feel confident with all three parts of a website, and I understand why problems like “the last seat” are so important in real systems. However, I think the most useful things I have learned are skills that I can use in any job.

- **Planning before doing.** Because I wrote down the nine rules before coding, I saved a lot of time. I always knew what I had to build next.
- **Learning by myself.** Some solutions, such as the lock, were not taught in my classes. I had to find the answers in guides and articles, and most of them were written in English.
- **Writing clearly.** I wrote guides so that other people could install and use the system. This taught me how to explain technical ideas in simple and clear language, which is also the goal of this report.
- **Managing my time.** Dividing the work into small stages helped me to make progress every week and to stay motivated.

If I had more time, I would add automatic tests, a waiting list for full classes with email messages, and a check to make sure that students have passed the courses which they need before joining a new one.

7. Conclusion

In conclusion, the Course Management and Registration System is the project that I am most proud of so far. It started from a problem that I had as a student, and it became a complete system that is fair, easy to use and easy to install. During the project, I faced some difficult problems, such as protecting the last seat in a class and finding timetable clashes. Solving them taught me to plan carefully, to learn by myself and to explain my ideas clearly.

Most importantly, this project has shown me that good software is not only about writing code. It is also about understanding people's problems and finding good solutions for them.